

2 – Alternativas Gratuitas de Autoridade Certificadora para Assinatura Digital de PDFs

Este documento complementa a justificativa anterior (seção 1) apresentando as principais alternativas gratuitas e de código aberto para implementação de uma Autoridade Certificadora (AC) local, com foco em assinatura digital de arquivos PDF em contexto acadêmico e de desenvolvimento. As opções estão organizadas em três categorias: bibliotecas Python, infraestruturas de PKI completas e ferramentas de linha de comando.

1. Por que não existe uma AC gratuita equivalente à ICP-Brasil

Uma Autoridade Certificadora reconhecida publicamente (como as credenciadas à ICP-Brasil) precisa, obrigatoriamente, passar por auditorias, manter infraestrutura física segura (HSMs certificados), operar sob legislação vigente e pagar taxas de credenciamento ao ITI. Esses requisitos tornam inviável a existência de uma AC gratuita com reconhecimento regulatório pleno.

Entretanto, para fins didáticos e de protótipo técnico — exatamente o escopo deste trabalho — existem soluções totalmente gratuitas e abertas que permitem montar uma hierarquia de certificação local (Root CA → Sub CA → certificado do assinante), assinar PDFs de forma criptograficamente válida e verificar a integridade dos documentos dentro do ecossistema da própria aplicação.

2. Opções gratuitas recomendadas

2.1 OpenSSL – Autoridade Certificadora local via linha de comando

O OpenSSL é a solução mais acessível para criar uma CA local completa sem qualquer custo. Permite gerar uma Root CA autoassinada, emitir certificados de entidade final (.p12/.pfx) e assinar PDFs utilizando as bibliotecas Python descritas adiante. É gratuito, open source (Apache 2.0) e disponível em todos os sistemas operacionais.

Fluxo resumido de criação da CA local com OpenSSL:

```
# 1. Gerar chave privada da Root CA openssl genrsa -out ca.key 4096 # 2. Criar
certificado autoassinado da Root CA (validade 10 anos) openssl req -new -x509 -days
3650 -key ca.key -out ca.crt \ -subj "/CN=Minha CA Local/O=Universidade/C=BR" # 3.
Gerar chave e CSR do assinante openssl genrsa -out assinante.key 2048 openssl req
-new -key assinante.key -out assinante.csr \ -subj "/CN=Nome do
Assinante/O=Instituicao/C=BR" # 4. Emitir certificado assinado pela CA openssl x509
-req -days 365 -in assinante.csr \ -CA ca.crt -CAkey ca.key -CAcreateserial -out
assinante.crt # 5. Empacotar em PKCS#12 (.pfx) para uso com bibliotecas Python
openssl pkcs12 -export -out assinante.p12 \ -inkey assinante.key -in assinante.crt
-certfile ca.crt
```

Documentação: <https://www.openssl.org/docs/man3.0/man1/>

2.2 pyHanko – Assinatura de PDFs em Python (MIT License)

pyHanko é a biblioteca Python mais completa e ativamente mantida para assinatura digital de PDFs. Suporta os perfis PAdES (padrão europeu), CAdES, assinaturas com carimbo de tempo (TSA/RFC 3161), múltiplas assinaturas, LTV (Long-Term Validation) e integração com tokens PKCS#11. Trabalha nativamente com certificados gerados pelo OpenSSL (arquivos .p12 ou .pem). É totalmente gratuita e de código aberto (licença MIT).

Instalação e exemplo de uso básico:

```
pip install 'pyHanko[pkcs11,image-support,opentype,qr]' pyhanko-cli # Assinatura via linha de comando pyhanko sign addsig --field Sig1 pemder \ --key assinante.key --cert assinante.crt \ documento.pdf documento_assinado.pdf # Ou via API Python: from pyhanko.sign import signers, fields from pyhanko.sign.fields import SigFieldSpec from pyhanko_certvalidator import ValidationContext from pyhanko.pdf_utils.incremental_writer import IncrementalPdfFileWriter with open('documento.pdf', 'rb') as inf: w = IncrementalPdfFileWriter(inf) signer = signers.SimpleSigner.load( 'assinante.key', 'assinante.crt', ca_chain_files=('ca.crt',), key_passphrase=b'senha' ) signers.sign_pdf( w, signers.PdfSignatureMetadata(field_name='Sig1'), signer=signer, output=open('saida.pdf', 'wb') )
```

Repositório: <https://github.com/MatthiasValvekens/pyHanko> | **Documentação:** <https://docs.pyhanko.eu>

2.3 endesive – Assinatura CAdES/PAdES em Python (MIT License)

endesive é uma biblioteca Python para assinatura e verificação de assinaturas digitais em PDFs, e-mails (S/MIME) e arquivos XML. Implementa o padrão CAdES-B no formato Adobe PPKLite/adbe.pkcs7.detached, além de suporte a XAdES BES/T. Também depende apenas de certificados X.509 padrão, compatíveis com os gerados pelo OpenSSL.

Instalação:

```
pip install endesive # Exemplo de assinatura de PDF: from endesive.pdf import cms from cryptography.hazmat.primitives.serialization import pkcs12 with open('assinante.p12', 'rb') as f: p12 = pkcs12.load_key_and_certificates(f.read(), b'senha') date = datetime.utcnow().strftime('D:%Y%m%d%H%M%S+00\00\') dct = {'sigflags': 3, 'sigpage': 0, 'sigbutton': True, 'contact': 'email@instituicao.br', 'location': 'Brasil', 'signingdate': date, 'reason': 'Assinatura Academica'} with open('documento.pdf', 'rb') as f: pdf_data = f.read() signed = cms.sign(pdf_data, dct, p12[0], p12[1], p12[2], 'sha256') with open('documento_assinado.pdf', 'wb') as f: f.write(pdf_data) f.write(signed)
```

PyPI: <https://pypi.org/project/endesive/> | **Repositório:** <https://github.com/m32/endesive>

2.4 EJBCA Community – PKI completo open source (LGPL v2.1)

O EJBCA (Enterprise JavaBeans Certificate Authority) é uma das soluções de PKI open source mais maduras do mundo, com mais de 20 anos de desenvolvimento. A edição Community é gratuita, licenciada sob LGPL v2.1, e disponível via Docker Hub, GitHub e SourceForge. Permite criar hierarquias de CA completas, gerenciar ciclo de vida de certificados (emissão, renovação, revogação), expor endpoints CRL e OCSP, e integrar via REST API com scripts Python.

Por ser uma solução Java/JEE com interface web administrativa, o EJBCA Community é indicado para cenários onde se deseja simular uma infraestrutura de PKI institucional completa — por exemplo, em trabalhos que envolvem múltiplos emissores de diplomas ou cadeia de confiança hierárquica.

Instalação via Docker (forma mais rápida):

```
docker run -it --rm -p 80:8080 -p 443:8443 \ -h minha-ca.local keyfactor/ejbca-ce
```

GitHub: <https://github.com/Keyfactor/ejbca-ce> | **Documentação:** <https://doc.primekey.com/ejbca> | **REST API Python:** <https://www.ejbca.org/case/automated-certificate-issuing-via-ejbca-rest/>

2.5 SignServer Community – Motor de assinatura de documentos (LGPL v2.1)

SignServer é um complemento natural ao EJBCA: enquanto o EJBCA emite certificados, o SignServer é responsável pela operação de assinatura em si, armazenando chaves de forma

centralizada e segura. Também é gratuito na edição Community, open source (LGPL), e disponível via Docker. Suporta assinatura de PDFs, código-fonte, timestamps e containers.

Site oficial: <https://www.signserver.org/> | Docker Hub: [keyfactor/signserver-ce](https://hub.docker.com/r/keyfactor/signserver-ce)

3. Quadro comparativo das alternativas

Ferramenta	Tipo	Linguagem	Ideal para	Licença
OpenSSL	CLI / Biblioteca	C	Gerar certificados X.509 e CA local	Apache 2.0
pyHanko	Biblioteca Python	Python	Assinar PDFs com PAdES/CAAdES, LTV	MIT
endesive	Biblioteca Python	Python	Assinatura CAAdES-B em PDFs e e-mails	MIT
EJBCA CE	PKI completo (Web)	Java	PKI institucional, múltiplos emissores	LGPL v2.1
SignServer CE	Motor de assinatura	Java	Assinatura centralizada e automatizada	LGPL v2.1

4. Conclusão e recomendação para o presente trabalho

Para o contexto deste trabalho acadêmico, a combinação mais indicada é: OpenSSL para gerar a hierarquia de CA local (Root CA + certificado do assinante em formato .p12) e pyHanko para aplicar a assinatura digital nos PDFs de diploma. Ambas as ferramentas são gratuitas, amplamente documentadas, e suficientes para demonstrar o funcionamento técnico completo do fluxo de assinatura eletrônica.

Conforme já detalhado na seção 1 deste documento, a ausência de reconhecimento pelo verificador do ITI (ICP-Brasil) não invalida a demonstração técnica: ela decorre exclusivamente do fato de o certificado utilizado não pertencer a uma cadeia pública credenciada, e não de qualquer falha criptográfica na assinatura produzida. Em ambiente de produção institucional, o mesmo fluxo seria mantido, substituindo-se apenas o certificado de teste por um emitido por AC credenciada à ICP-Brasil.

Este documento complementa: *1 – Justificativa de impossibilidade de utilização de certificado ICP-Brasil válido.*